



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

Department of Electronics and Telecommunication Engineering

Name: Janay Rawal Prn:22070123049 Batch: ENTC 22-26

EXPERIMENT NO. 2:

AIM:

To implement the Hough Transform for Circle Detection.

Software Required: Python and VS code

THEORY

The Hough Transform is a widely used feature extraction technique in image analysis, computer vision, and digital image processing. It is particularly effective for detecting geometric shapes such as lines, circles, and ellipses in an image, even in the presence of noise or partial occlusion.

Hough Transform for Circle Detection

A circle in a 2D space can be defined by the equation:

$$(x-a)^2+(y-b)^2=r^2$$

where:

- (x,y) are the coordinates of a point on the circle.
- (a,b) are the coordinates of the center of the circle. r is the radius of the circle.

Steps Involved:

1. **Edge Detection:**
The input image is first converted to grayscale and processed using an edge detection algorithm like the Canny Edge Detector. This helps in identifying potential boundary points of circles.
2. **Voting in the Accumulator Space:**
For each edge pixel (x,y), the algorithm votes for all possible circle centers (a,b) for a given set of radii r. This is done using the circle equation rearranged for center coordinates.
3. **Peak Detection:**
The points in the accumulator space with the highest votes indicate the most probable circle parameters. These peaks correspond to detected circles in the image.
4. **Drawing Circles:**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

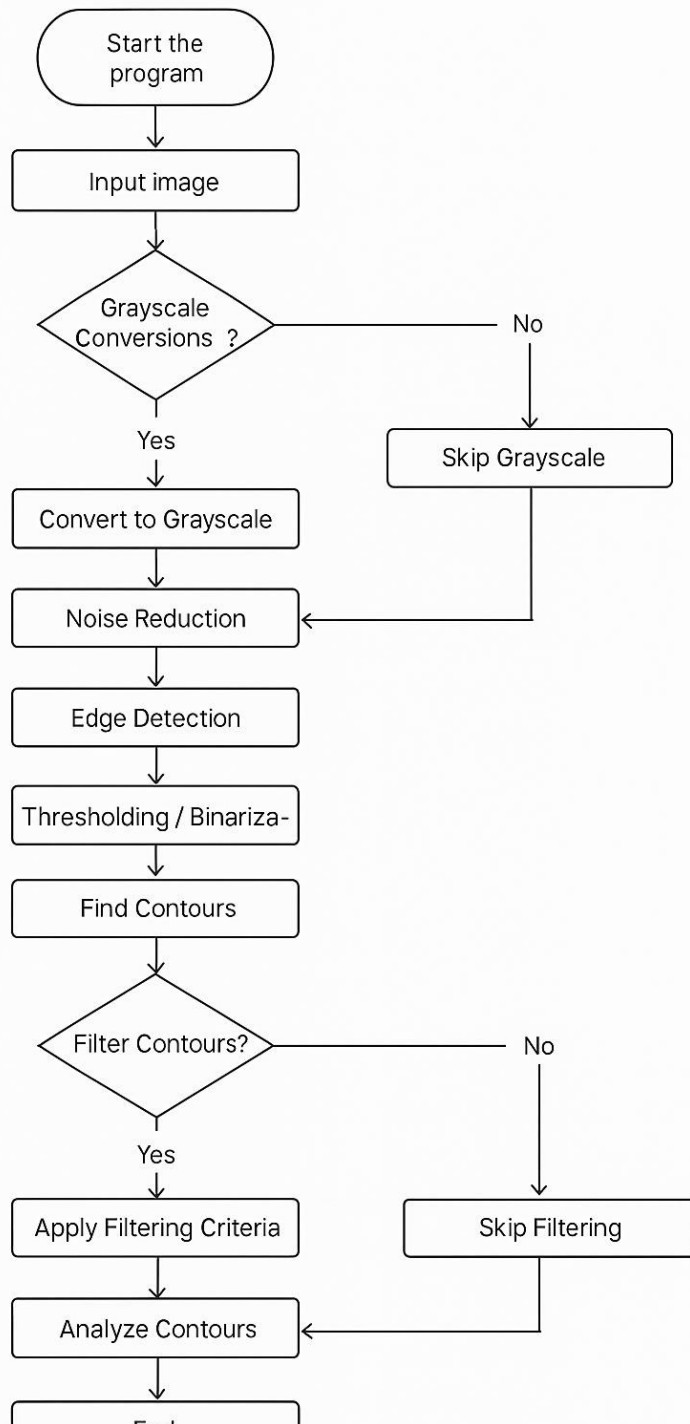
Finally, circles corresponding to high-scoring parameters are drawn on the original image.

Department of Electronics and Telecommunication Engineering

Flowchart: 2A

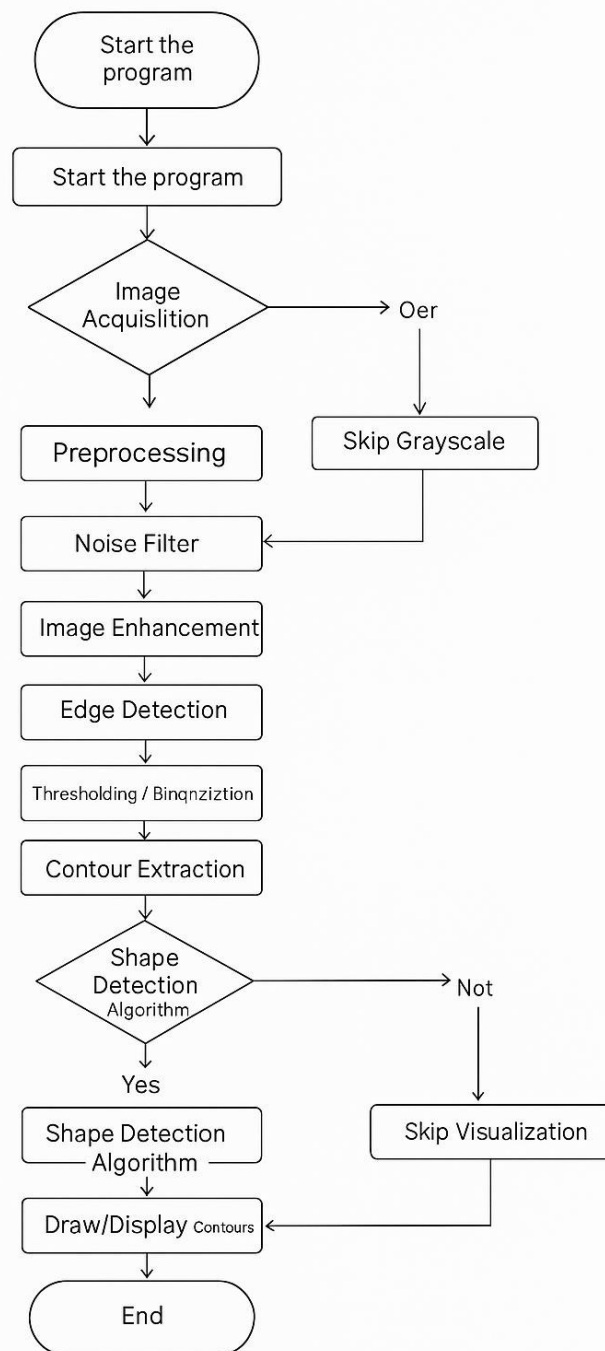


SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune



Department of Electronics and Telecommunication Engineering

Code: 2A



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
C: > Users > romil > Downloads > import cv2.py > ...
1  import cv2
2  import numpy as np
3
4  #
5  # CALLBACK FUNCTION FOR TRACKBAR (DO NOTHING)
6  #
7  def nothing(x):
8      pass
9
10 #
11 # LOAD IMAGE
12 #
13 IMAGE_PATH = "coins.png"
14 orig = cv2.imread(IMAGE_PATH)
15 if orig is None:
16     raise FileNotFoundError(f"Cannot open {IMAGE_PATH}")
17 gray = cv2.cvtColor(orig, cv2.COLOR_BGR2GRAY)
18
19 #
20 # CREATE WINDOW AND TRACKBARS
21 #
22 cv2.namedWindow("Detected Circles")
23
24 # Parameters for trackbars (renamed & reordered)
25 cv2.createTrackbar("Canny Th", "Detected Circles", 100, 255, nothing)
26 cv2.createTrackbar("Hough Param2", "Detected Circles", 30, 100, nothing)
27 cv2.createTrackbar("MinDist", "Detected Circles", 40, 200, nothing)
28 cv2.createTrackbar("Min Radius", "Detected Circles", 10, 100, nothing)
29 cv2.createTrackbar("Max Radius", "Detected Circles", 80, 200, nothing)
30
31 #
32 # DYNAMIC LOOP
33 #
34 while True:
35     # Read trackbar values
36     canny_th = cv2.getTrackbarPos("Canny Th", "Detected Circles")
37     param2 = cv2.getTrackbarPos("Hough Param2", "Detected Circles")
38     min_dist = cv2.getTrackbarPos("MinDist", "Detected Circles")
39     min_r = cv2.getTrackbarPos("Min Radius", "Detected Circles")
40     max_r = cv2.getTrackbarPos("Max Radius", "Detected Circles")
41
42     # Preprocessing
43     blurred = cv2.GaussianBlur(gray, (9, 9), sigmaX=2, sigmaY=2)
44     edges = cv2.Canny(blurred, canny_th // 2, canny_th) # use low/high from same value
45
46     # Hough Circle Transform
47     circles = cv2.HoughCircles(
48         blurred,
49         cv2.HOUGH_GRADIENT,
50         dp=1.2,
51         minDist=max(1, min_dist),
52         param1=canny_th,
53         param2=param2,
54         minRadius=min_r,
55         maxRadius=max_r
56     )
57
58     # Draw result
59     output = orig.copy()
60     if circles is not None:
61         circles = np.uint16(np.around(circles[0]))
62         for (x, y, r) in circles:
63             cv2.circle(output, (x, y), r, (0, 255, 0), 2)
64             cv2.circle(output, (x, y), 2, (0, 0, 255), 3)
65
66     # Show images
67
68     cv2.imshow("Detected Circles", output)
69     cv2.imshow("Edges", edges)
70
71     key = cv2.waitKey(50) & 0xFF
72     if key == 27: # ESC to exit
73         break
74
75 cv2.destroyAllWindows()
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Department of Electronics and Telecommunication Engineering 2B:

```
1 import cv2
2 import numpy as np
3
4 #
5 # Tracker Callback
6 #
7 def nothing(x):
8     pass
9
10 #
11 # Shape Detection by Approximation
12 #
13 def detect_shape(approx, contour):
14     sides = len(approx)
15     if sides == 3:
16         return "Triangle"
17     elif sides == 4:
18         (x, y, w, h) = cv2.boundingRect(approx)
19         aspect_ratio = w / float(h)
20         if 0.95 <= aspect_ratio <= 1.05:
21             return "Square"
22         else:
23             angles = []
24             for i in range(4):
25                 p1 = approx[i][0]
26                 p2 = approx[(i + 1) % 4][0]
27                 p3 = approx[(i + 2) % 4][0]
28                 v1 = p1 - p2
29                 v2 = p3 - p2
30                 angle = np.degrees(np.arccos(np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))))
31                 angles.append(angle)
32             if np.allclose(angles, 90, atol=10):
33                 return "Rectangle"
34             elif np.isclose(angles[0], angles[2], atol=10):
35                 return "Rhombus"
36             else:
```

```
37         return "Trapezoid"
38     elif sides == 5:
39         return "Pentagon"
40     elif sides == 6:
41         return "Hexagon"
42     elif 8 <= sides <= 10:
43         return "Star"
44     elif sides > 10:
45         area = cv2.contourArea(contour)
46         perimeter = cv2.arcLength(contour, True)
47         if perimeter == 0:
48             return "Unknown"
49         circularity = 4 * np.pi * area / (perimeter * perimeter)
50         if circularity > 0.8:
51             return "Circle"
52         return "Star Polygon"
53     else:
54         return "Polygon"
55
56 #
57 # Load and Preprocess Image
58 #
59 image = cv2.imread("shapes.jpg") # Make sure this image is in the same folder
60 resized = cv2.resize(image, (600, 600))
61 gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
62 blurred = cv2.medianBlur(gray, 5)
63
64 #
65 # Create Fullscreen Window and Trackbars
66 #
67 cv2.namedWindow("Result", cv2.WINDOW_NORMAL)
68 cv2.setWindowProperty("Result", cv2.WND_PROP_FULLSCREEN, cv2.WINDOW_FULLSCREEN)
69
70 cv2.createTrackbar("Hough Param1", "Result", 100, 300, nothing)
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
71 cv2.createTrackbar("Hough Param2", "Result", 30, 100, nothing)
72 cv2.createTrackbar("Min Radius", "Result", 10, 100, nothing)
73 cv2.createTrackbar("Max Radius", "Result", 100, 200, nothing)
74
75 while True:
76     output = resized.copy()
77     total_shapes = 0
78
79     # =====
80     # Circle Detection (Hough)
81     # =====
82     p1 = cv2.getTrackbarPos("Hough Param1", "Result")
83     p2 = cv2.getTrackbarPos("Hough Param2", "Result")
84     minR = cv2.getTrackbarPos("Min Radius", "Result")
85     maxR = cv2.getTrackbarPos("Max Radius", "Result")
86
87     circles = cv2.HoughCircles(blurred, cv2.HOUGH_GRADIENT, dp=1, minDist=30,
88                               param1=p1, param2=p2, minRadius=minR, maxRadius=maxR)
89
90     circle_centers = []
91     if circles is not None:
92         circles = np.uint16(np.around(circles))
93         for i in circles[0, :]:
94             cx, cy, r = i
95             circle_centers.append((cx, cy))
96             cv2.circle(output, (cx, cy), r, (0, 255, 0), 2)
97             cv2.putText(output, "Circle", (cx - 30, cy), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 100, 0), 2)
98             total_shapes += 1
99
100     # =====
101     # Contour Detection
102     # =====
103     edged = cv2.Canny(blurred, 50, 150)
104     contours, _ = cv2.findContours(edged, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

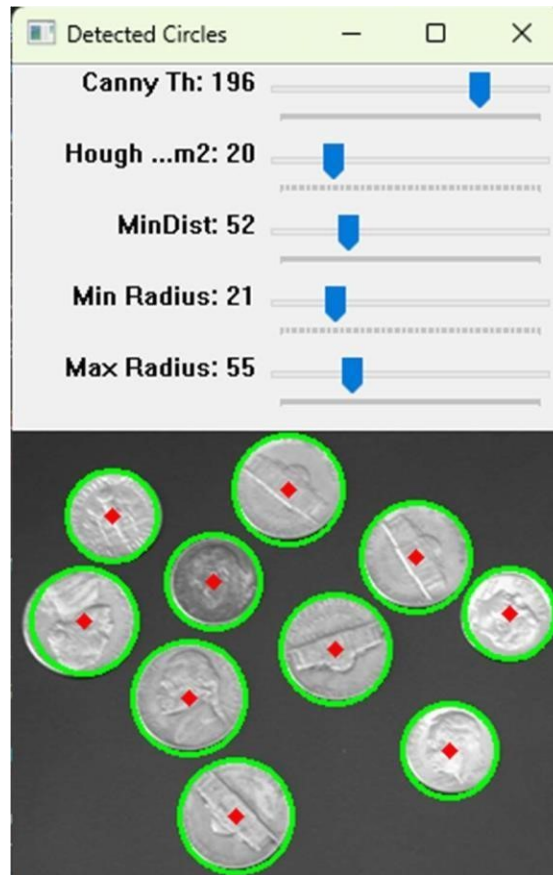
```
106     for cnt in contours:
107         area = cv2.contourArea(cnt)
108         if area > 500:
109             M = cv2.moments(cnt)
110             if M["m00"] == 0:
111                 continue
112             cx = int(M["m10"] / M["m00"])
113             cy = int(M["m01"] / M["m00"])
114
115             if any(np.linalg.norm(np.array([cx, cy]) - np.array(center)) < 20 for center in circle_centers):
116                 continue
117
118             approx = cv2.approxPolyDP(cnt, 0.02 * cv2.arcLength(cnt, True), True)
119             shape = detect_shape(approx, cnt)
120
121             cv2.drawContours(output, [approx], -1, (0, 0, 255), 2)
122             cv2.putText(output, shape, (cx - 40, cy), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 100, 0), 2)
123             total_shapes += 1
124
125     # =====
126     # Show Shape Count
127     # =====
128     cv2.putText(output, f"Total Shapes: {total_shapes}", (10, 590), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
129
130     # =====
131     # Display Result
132     # =====
133     cv2.imshow("Result", output)
134     if cv2.waitKey(1) & 0xFF == ord('q'):
135         break
136
137 cv2.destroyAllWindows()
```

Department of Electronics and Telecommunication Engineering

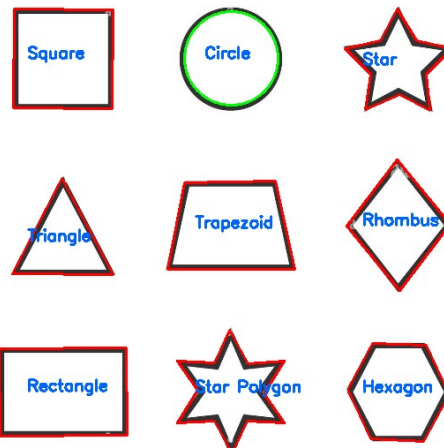
OUTPUT: 2A



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune



2B:



Total Shapes: 9

Department of Electronics and Telecommunication Engineering

CONCLUSION:



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE
Constituent of Symbiosis International (Deemed University), Pune

We have implemented the Hough Transform to successfully detect circles in an image of coins. The method accurately identified the circular shapes, demonstrating the effectiveness of the algorithm in real-world object detection using computer vision.